Міністерство освіти і науки України
Національний університет «Одеська політехніка»
Інститут комп'ютерних систем
Кафедра інформаційних систем

Протокол
Лабораторна робота №1
З дисципліни: «Програмування мобільних пристроїв на базі ОС Android»
Тема: «Основи мови Kotlin»

Виконала:
студентка групи АІ-231
Гетьманенко Л. О.

Перевірив:
Годовиченко М. А.

Одеса – 2026

Мета роботи. Ознайомлення з фундаментальними концепціями мови програмування Kotlin. Набуття практичних навичок оголошення змінних, використання рядкових шаблонів, керуючих конструкцій (if, when) та циклів (for, while). Формування компетенцій щодо написання, компіляції та тестування програмного коду у середовищі розробки.

Завдання

1. Опрацювати базовий синтаксис Kotlin (Basic Syntax): використання змінних (val, var), виведення типів та рядкові шаблони.
2. Дослідити систему типів та механізми безпеки від відсутності значення (Type System and Null Safety), включаючи безпечні виклики (?.) та Elvis-оператор (?:).
3. Застосувати керуючі конструкції (if, when як вирази) та цикли (for, робота з діапазонами) для побудови алгоритмів (Control Flow).
4. Ознайомитися зі способами оголошення функцій (Functions): використанням параметрів за замовчуванням, іменованих аргументів та функцій з одного виразу (Single-Expression).
5. Вивчити принципи об'єктно-орієнтованого програмування в Kotlin (Object-Oriented Programming in Kotlin): створення класів, конструктори, властивості з кастомними методами доступу, data та sealed класи.
6. Опрацювати роботу з колекціями та навчитися створювати функції-розширення (Collections and Extension Functions).
7. Засвоїти специфічний синтаксис Kotlin, необхідний для роботи з декларативним UI (Kotlin Syntax for Jetpack Compose): лямбда-вирази поза дужками (Trailing Lambdas), неявний параметр it та використання областей видимості (Scopes) для побудови ієрархічних структур.

Хід роботи

Тема 1. Basic Syntax

Задача 1. Розв'язок першої задачі з оголошення незмінної змінної реалізовано таким чином:

```
fun getLanguageName(): String {
    val language: String = "Kotlin"
    return language
}
```

Задача 2. Для зміни значення змінної в процесі виконання написано наступний код:

```
fun updateCounter(): Int {
    var counter: Int = 0
    counter += 5
    return counter
}
```

Задача 3. Демонстрація автоматичного виведення типів (Type Inference) компілятором:

```
fun getKotlinVersion(): Double {
    val version = 1.9
    return version
}
```

Тема 2. Type System and Null Safety

Задача 1. Для роботи зі стандартними (Non-Nullable) типами створено функцію:

```
fun getLength(text: String): Int {
    return text.length
}
```

Задача 2. Оголошення функції з Nullable-типом для можливості повернення значення null:

```
fun getNullValue(): String? {
    return null
```

```
}
```

Задача 3. Використання безпечного виклику ?. під час роботи з Nullable типом:

```
fun getLengthSafe(text: String?): Int? {
    return text?.length
}
```

Тема 3. Control Flow

Задача 1. Використання керуючої конструкції if як виразу, що безпосередньо повертає більше з двох значень:

```
fun getMax(a: Int, b: Int): Int {
    return if (a > b) a else b
}
```

Задача 2. Заміна класичного тернарного оператора за допомогою однорядкової конструкції if-else для перевірки на парність:

```
fun checkParity(number: Int): String {
    return if (number % 2 == 0) "Even" else "Odd"
}
```

Задача 3. Реалізація логіки множинного вибору (аналог switch) за допомогою базової конструкції when:

```
fun getNumberName(n: Int): String {
    return when (n) {
        1 -> "One"
        2 -> "Two"
        3 -> "Three"
        else -> "Unknown"
    }
}
```

Тема 4. Functions

Задача 1. Оголошення базової функції без передачі аргументів, яка повертає фіксоване рядкове значення:

```
fun getGreeting(): String {
    return "Hello, Kotlin!"
}
```

Задача 2. Створення функції з визначеними параметрами для обчислення та повернення добутку двох переданих цілих чисел:

```
fun multiply(a: Int, b: Int): Int {
    return a * b
}
```

Задача 3. Використання скороченого синтаксису (Single-Expression) для лаконічного запису функції в один рядок:

```
fun square(x: Int) = x * x
```

Тема 5. Object-Oriented Programming in Kotlin

Задача 1. Створення класу з первинним конструктором та оголошенням його властивостей, а також зміна стану створеного об'єкта:

```
class Student(val name: String, var age: Int)

fun createStudent(): String {
    val student = Student("Ivan", 20)
    student.age = 21
    return "${student.name} is ${student.age}"
}
```

Задача 2. Використання блоку ініціалізації (init) для перевірки вхідних параметрів та генерації винятку під час створення екземпляра класу:

```
class Product(val price: Double) {
    init {
        if (price < 0) {
            throw    IllegalArgumentException("Invalid    price:
$price. Price cannot be negative")
        }
    }
}
```

Задача 3. Оголошення вторинного конструктора за допомогою ключового слова constructor та делегування виклику первинному конструктору:

```
class Rectangle(val width: Int, val height: Int) {
    constructor(size: Int) : this(size, size)
}

fun createSquare(): Int {
    val square = Rectangle(5)
    return square.width * square.height
}
```

Тема 6. Collections and Extension Functions

Задача 1. Створення базового незмінного списку (read-only) за допомогою стандартної функції ініціалізації:

```
fun getFruits(): List<String> {
    return listOf("Apple", "Banana", "Orange")
}
```

Задача 2. Робота зі змінною колекцією (MutableList) та динамічне додавання нових елементів після її створення:

```
fun createMutableList(): MutableList<Int> {
    val list = mutableListOf(1, 2)
    list.add(3)
    return list
}
```

Задача 3. Безпечне приведення типів: формування змінної колекції всередині функції та повернення її назовні як незмінної (List):

```
fun getReadOnlyList(): List<String> {
    val list = mutableListOf("A", "B", "C")
    list.add("D")
    return list
}
```

Тема 7. Kotlin Syntax for Jetpack Compose

Задача 1. Оголошення функції верхнього рівня з використанням конвенції PascalCase для опису UI-елемента:

```
fun Text(text: String): String {
    return "[Text: $text]"
}
```

Задача 2. Створення екземпляра класу без використання ключового слова new та передача його як аргументу функції:

```
class Modifier(val id: String)
fun Box(modifier: Modifier): String = "Box with ${modifier.id}"

fun createBox(): String {
    return Box(Modifier("ID_1"))
}
```

Задача 3. Виклик функції з використанням іменованих аргументів для зручного налаштування параметрів та пропуску значень за замовчуванням:

```
fun MyText(text: String, color: String = "Black", fontSize: Int
= 14): String {
    return "Text: $text, Color: $color, Size: $fontSize"
}

fun callText(): String {
    return MyText(text = "Compose", fontSize = 24)
}
```

Висновки

Під час виконання лабораторної роботи було успішно засвоєно базовий синтаксис та фундаментальні концепції мови програмування Kotlin. Отримано практичні навички роботи з керуючими конструкціями, функціями, об'єктно-орієнтованим підходом та колекціями. Особливу увагу було приділено механізмам безпеки типів (Null Safety) та специфічним синтаксичним конструкціям (Trailing Lambdas, Scopes), які є необхідною базою для побудови декларативних інтерфейсів у Jetpack Compose. Отримані знання стануть базою для подальшого вивчення Android-розробки.